

Programming Language (2)

Object Oriented Programming



Introduction to
JAVATM
PROGRAMMING AND
DATA STRUCTURES

COMPREHENSIVE VERSION



Y. DANIEL LIANG



12th Edition

Introduction to Java Programming and Data Structures, Comprehensive Version, 12th Edition

Y. Daniel Liang, Georgia Southern University

©2020 | Pearson | Available

Grades Distribution

1)Final Written Exam	50%
2)Midterm Written Exam	20%
3)Sheets	10%
4)Quizzes	10%
5)Project	10%

Course Materials & Announcements

Announcements, and any supplemental notes will be made available “posted” online on the: **MS Teams Group with code: m3wc9uc**

Introduction to *JAVA*

Java

- Java was developed by a team led by James Gosling at Sun Microsystems. Sun Microsystems was purchased by Oracle in 2010.
 - Java was designed in 1991 for use in embedded chips, originally called Oak. In 1995, renamed Java.
- Java is a **general-purpose programming language**.
 - Enables users to develop and deploy applications for servers, desktop computers, and small hand-held devices.
- Java is the **Internet programming language**.

Programs

Computer *programs* are set of instructions. You tell a computer what to do through programs. Without programs, a computer is an empty machine.

Computers do not understand human languages, so you need to use computer languages to communicate with them.

Programs are written using programming languages.

Programs

A program written in a high-level language is called a *source program* or *source code*.

Because a computer cannot understand a source program, it **must be translated into machine code for execution**.

A statement from the source code may be translated into several machine instructions.

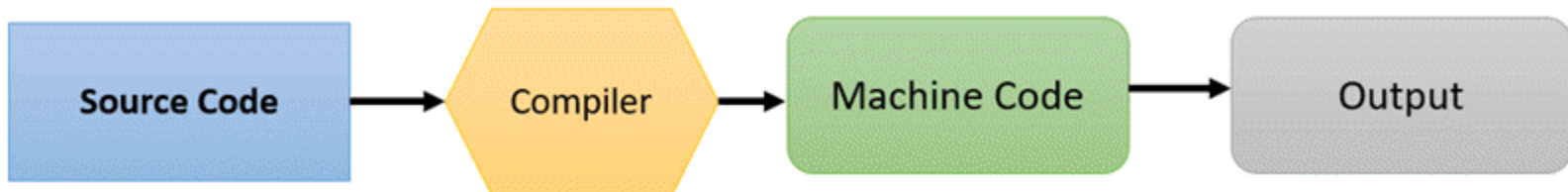
The translation can be done using another programming tool called an *interpreter* or a *compiler*.

Program

An **interpreter** reads one statement from the source code, translates it to the machine code and then executes it right away.

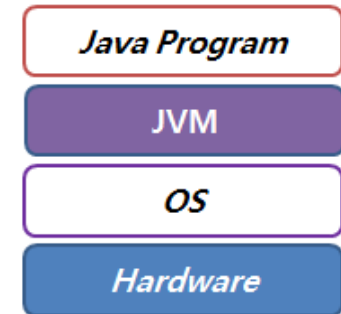
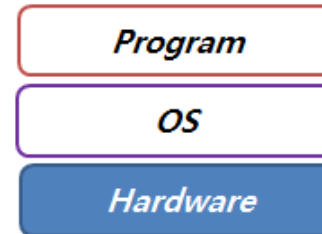
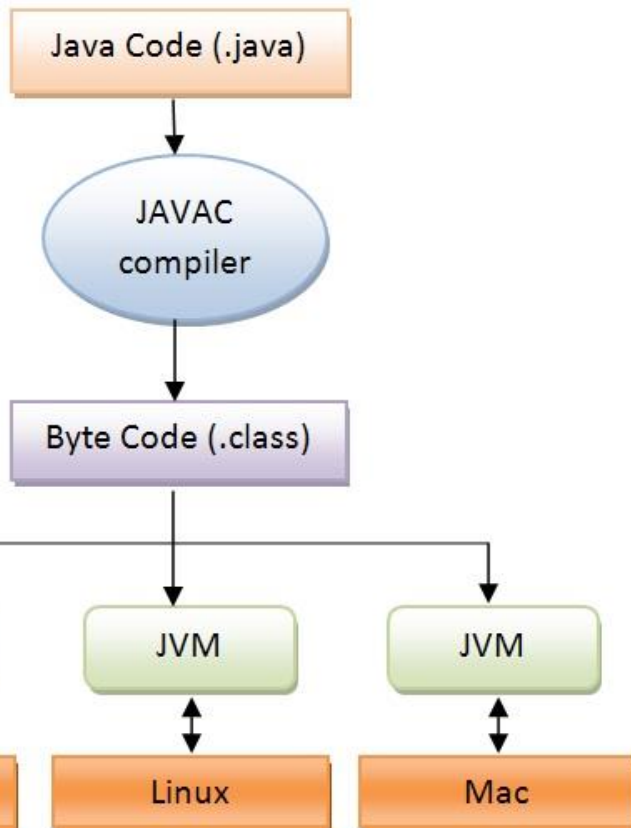
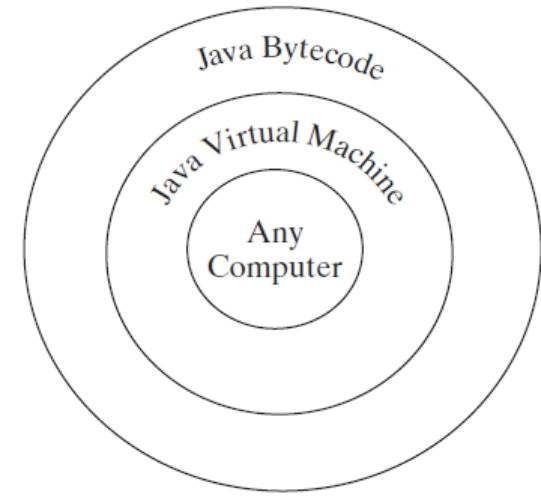


A **compiler** translates the entire source code into a machine-code file, and the machine-code file is then executed *only run on a specific machine*.



Java Program

With Java, you write the program once, and compile the source program into a special type of object code, known as *bytecode*. The bytecode is similar to machine instructions but can run on any computer with a **Java Virtual Machine (JVM)**.



JVM is a software that interprets Java bytecode. It is available for many different platforms.

Characteristics of Java

- ❑ Java Is Simple
- ❑ Java Is Object-Oriented
- ❑ Java Is Distributed, Interpreted, Portable, Dynamic
- ❑ Java Is Robust, Secure
- ❑ Java's Performance
- ❑ Java Is Multithreaded

JDK Editions

- Java Standard Edition (J2SE)
 - J2SE can be used to develop client-side standalone applications or applets.
- Java Enterprise Edition (J2EE)
 - J2EE can be used to develop server-side applications such as Java servlets, Java ServerPages, and Java ServerFaces.
- Java Micro Edition (J2ME).
 - J2ME can be used to develop applications for mobile devices such as cell phones.

Simple *JAVA* Program

```
File Edit Format View Help
// This application program prints Welcome to Java!
public class Welcome {
    public static void main(String[] args) {
        System.out.println("Welcome to Java!");
    }
}
```

Creating, Compiling, and Running Programs

Source code (developed by the programmer)

```
public class Welcome {
    public static void main(String[] args) {
        System.out.println("Welcome to Java!");
    }
}
```

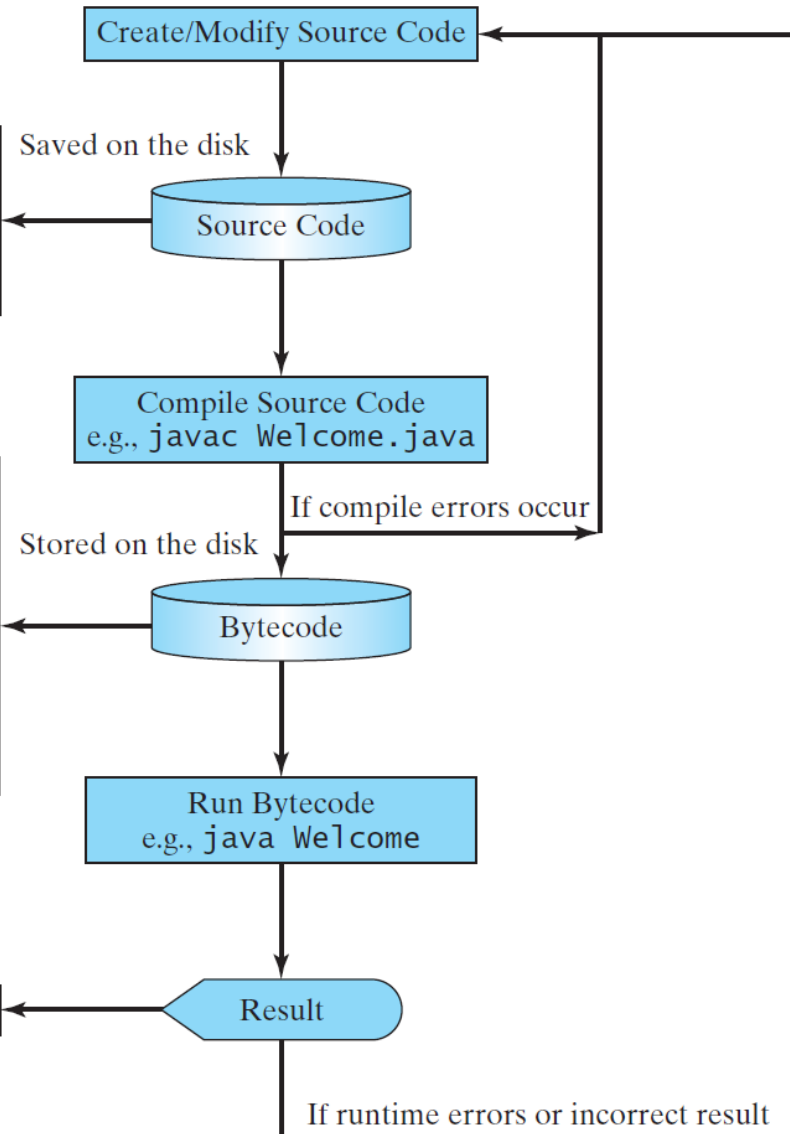
Bytecode (generated by the compiler for JVM to read and interpret)

```
...
Method Welcome()
  0 aload_0
  ...

Method void main(java.lang.String[])
  0 getstatic #2 ...
  3 ldc #3 <String "Welcome to Java!">
  5 invokevirtual #4 ...
  8 return
```

“Welcome to Java” is displayed on the console

```
Welcome to Java!
```



A Simple Java Program

Listing 1.1

```
// This program prints Welcome to Java!
public class Welcome {
    public static void main(String[] args) {
        System.out.println("Welcome to Java!");
    }
}
```

Welcome

Note: Clicking the green button displays the source code with interactive animation. You can also run the code in a browser. Internet connection is needed for this button.

Class Name

Every Java program must have at least one class. Each class has a name. By convention, class names start with an uppercase letter.

```
// This program prints Welcome to Java!
public class Welcome {
    public static void main(String[] args) {
        System.out.println("Welcome to Java!");
    }
}
```


Main Method

Line 2 defines the main method. In order to run a class, the class must contain a method named main. The program is executed from the main method.

```
// This program prints Welcome to Java!  
public class Welcome {  
    public static void main(String[] args) {  
        System.out.println("Welcome to Java!");  
    }  
}
```

Statement

A statement represents an action or a sequence of actions.

The statement `System.out.println("Welcome to Java!")` displays the greeting "Welcome to Java!".

```
// This program prints Welcome to Java!  
public class Welcome {  
    public static void main(String[] args) {  
        System.out.println("Welcome to Java!");  
    }  
}
```

Keywords

Keywords are words that have a specific meaning to the compiler and cannot be used for other purposes in the program. For example, when the compiler sees the word `class`, it understands that the word after `class` is the name for the class.

```
// This program prints Welcome to Java!  
public class Welcome {  
    public static void main(String[] args) {  
        System.out.println("Welcome to Java!");  
    }  
}
```

Programming Style and Documentation

- Appropriate Comments
- Naming Conventions
- Proper Indentation and Spacing Lines
- Block Styles

Programming Style and Documentation

□ Appropriate Comments

- In addition to line comments (beginning with `//`) and block comments (beginning with `/*`), Java supports comments of a special type, referred to as *javadoc comments*, begin with `/**` and end with `*/`. They can be extracted into an HTML file using the JDK's **javadoc** command.

□ Naming Conventions

□ Proper Indentation and Spacing Lines

□ Block Styles

Programming Style and Documentation

- Appropriate Comments

- Naming Conventions

- Choose meaningful and descriptive names.
- Variables and method names:
 - Use lowercase. If the name consists of several words, concatenate all in one, use lowercase for the first word, and capitalize the first letter of each subsequent word in the name. For example, the variables `radius` and `area`, and the method `computeArea`.

- Proper Indentation and Spacing Lines

- Block Styles

Programming Style and Documentation

- Appropriate Comments

- Naming Conventions

 - Class names:

 - Capitalize the first letter of each word in the name. For example, the class name `ComputeArea`.

 - Constants:

 - Capitalize all letters in constants and use underscores to connect words. For example, the constant `PI` and `MAX_VALUE`

- Proper Indentation and Spacing Lines

- Block Styles

Programming Errors

- Syntax Errors
 - Detected by the compiler
- Runtime Errors
 - Causes the program to abort
- Logic Errors
 - Produces incorrect result

Elementary Programming

Introducing Programming with an Example

Listing 2.1 Computing the Area of a Circle

This program computes the area of the circle.

ComputeArea

Note: Clicking the green button displays the source code with interactive animation. You can also run the code in a browser. Internet connection is needed for this button.

Reading Input from the Console

1. Create a Scanner object

```
Scanner input = new Scanner(System.in);
```

2. Use the method nextDouble() to obtain to a double value. For example,

```
System.out.print("Enter a double value: ");  
Scanner input = new Scanner(System.in);  
double d = input.nextDouble();
```

ComputeAreaWithConsoleInput

ComputeAverage

Implicit Import and Explicit Import

```
java.util.* ; // Implicit import
```

```
java.util.Scanner; // Explicit Import
```

No performance difference

Assignment Statements

□ `x = 1;` `// Assign 1 to x;`

□ `System.out.println( x = 3 );`

equivalent to:

`x = 3;` `System.out.println( x );`

□ `x = y = z = 1;`

equivalent to:

`x = 1;` `y = 1;` `z = 1;`

Named Constants

```
final datatype CONSTANTNAME = VALUE;
```

EX:

```
final double PI = 3.14159;
```

```
final int SIZE = 3;
```

Numerical Data Types

Name	Range	Storage Size
Byte	-2^7 to $2^7 - 1$ (-128 to 127)	8-bit signed
short	-2^{15} to $2^{15} - 1$ (-32768 to 32767)	16-bit signed
int	-2^{31} to $2^{31} - 1$ (-2147483648 to 2147483647)	32-bit signed
long	-2^{63} to $2^{63} - 1$ (i.e., -9223372036854775808 to 9223372036854775807)	64-bit signed
float	Negative range: -3.4028235E+38 to -1.4E-45 Positive range: 1.4E-45 to 3.4028235E+38	32-bit IEEE 754
double	Negative range: -1.7976931348623157E+308 to -4.9E-324 Positive range: 4.9E-324 to 1.7976931348623157E+308	64-bit IEEE 754

Note: *Calculations involving floating-point numbers are approximated because these numbers are not stored with complete accuracy.*

EX: `System.out.println(1.0 - 0.1 - 0.1 - 0.1 - 0.1 - 0.1);`
displays `0.5000000000000001`, not `0.5`.

Reading Numbers from the Keyboard

```
Scanner input = new Scanner(System.in);  
int value = input.nextInt();
```

Method	Description
<code>nextByte()</code>	reads an integer of the <code>byte</code> type.
<code>nextShort()</code>	reads an integer of the <code>short</code> type.
<code>nextInt()</code>	reads an integer of the <code>int</code> type.
<code>nextLong()</code>	reads an integer of the <code>long</code> type.
<code>nextFloat()</code>	reads a number of the <code>float</code> type.
<code>nextDouble()</code>	reads a number of the <code>double</code> type.

If you enter a value with an incorrect range or format, a runtime error would occur.

Number Literals

A *literal* is a constant value that appears directly in the program.

EX: 34 14002300 and 5.0

To improve readability, Java allows you to use underscores to separate two digits in a number literal.

EX: 14_002_300 1_234_554.211_234

By default, a floating-point literal is treated as a double type value.

➔ float f = 5.0 *is a syntax error*

A number is made a float by appending the letter **f** or **F**.

EX: float avg = 5.0F

Scientific Notation

Floating-point literals can also be specified in scientific notation, where the character **e** or **E** denotes the fact that the number is to be multiplied by the power of ten of the integer following the e or E

EX: `double y = 1.43E8;`
`double z = 1e-4;`

JShell

JShell is a command line interactive tool introduced in Java 9. JShell enables you to type a single Java statement and get it executed to see the result right away without having to write a complete class. This feature is commonly known as REPL (Read-Evaluate-Print Loop), which evaluates expressions and executes statements as they are entered and shows the result immediately.

Thank you
for your donation

